

Logistic Regression

April 11, 2025

0.0.1 Libraries

```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn import metrics
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import precision_recall_curve, average_precision_score
from sklearn.metrics import PrecisionRecallDisplay
```

0.0.2 Load Dataset

```
[2]: # Load Data
df_original = pd.read_csv('df.csv')
df_original['Malnurished'] = df_original[['Underweight', 'Stunting',
↳ 'Wasting']].max(axis=1)
df = df_original.drop(['Underweight', 'Stunting', 'Wasting'], axis = 1)

# Train-test Split
X = df.drop(columns=['Malnurished'])
y = df['Malnurished']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20,
↳ stratify = y, random_state = 42)
```

```
[3]: print(df.shape)
print(y_train.value_counts())
print(y_test.value_counts())
```

```
(2205, 37)
Malnurished
0    1168
1     596
Name: count, dtype: int64
```

```
Malnourished
0      292
1      149
Name: count, dtype: int64
```

Standard Scaler

```
[4]: columns_to_scale = ['Child_age', 'BMI', 'Mother_age_current']
     scaler = StandardScaler()

     # X_train_scaled = X_train.copy()
     # X_test_scaled = X_test.copy()

     X_train[columns_to_scale] = scaler.fit_transform(X_train[columns_to_scale])
     X_test[columns_to_scale] = scaler.transform(X_test[columns_to_scale])
```

0.1 Model Fit

```
[5]: lr = LogisticRegression(class_weight='balanced', penalty=None, random_state=42)
     lr.fit(X_train, y_train)
     y_pred = lr.predict(X_test)

     print(f'Confusion Matrix: \n{pd.crosstab(y_test, y_pred)}\n')
     print(f'Classification Report: \n{classification_report(y_test, y_pred)}\n')
```

Confusion Matrix:

col_0	0	1
Malnourished		
0	171	121
1	52	97

Classification Report:

	precision	recall	f1-score	support
0	0.77	0.59	0.66	292
1	0.44	0.65	0.53	149
accuracy			0.61	441
macro avg	0.61	0.62	0.60	441
weighted avg	0.66	0.61	0.62	441

class_weight='balanced' is used instead of SMOTE for balancing the imbalanced classes

0.1.1 Hyperparameter Tunning

```
[6]: param_grid = {
      'C': [0.001, 0.01, 0.1, 1, 10, 100],
      'penalty': ['l1', 'l2'],
      'solver': ['liblinear']
    }

    # Set up GridSearchCV with recall as the scoring metric
    grid_search = GridSearchCV(estimator=lr,
                              param_grid=param_grid,
                              cv=5,
                              scoring='recall', # Optimize for recall
                              n_jobs=-1,
                              verbose=1)

    # Fit GridSearchCV on the scaled training data
    grid_search.fit(X_train, y_train)

    # Output best parameters based on recall
    print("Best Hyperparameters:", grid_search.best_params_)
```

Fitting 5 folds for each of 12 candidates, totalling 60 fits

Best Hyperparameters: {'C': 0.001, 'penalty': 'l2', 'solver': 'liblinear'}

Model Fit With Best Parameters

```
[7]: # Use the best estimator to make predictions on the test set
      lr_best = grid_search.best_estimator_
      probas = lr_best.predict(X_test)

      print(classification_report(y_test, probas))
```

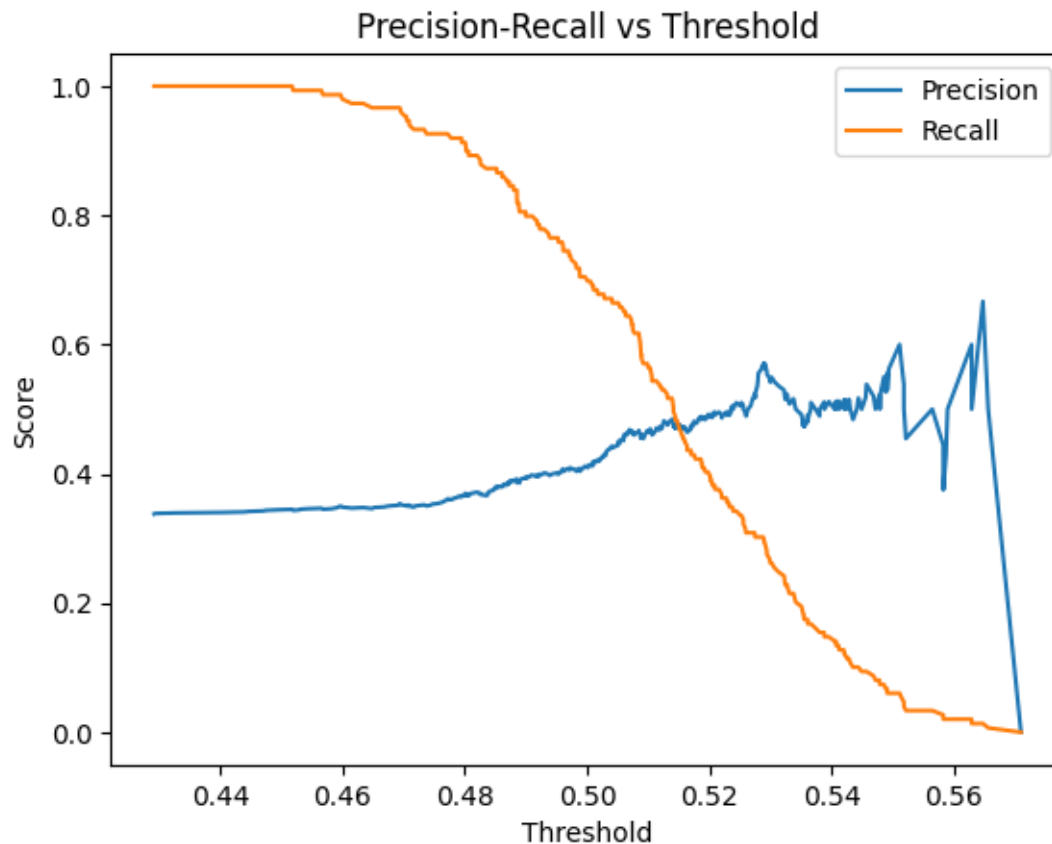
	precision	recall	f1-score	support
0	0.76	0.49	0.59	292
1	0.41	0.70	0.52	149
accuracy			0.56	441
macro avg	0.58	0.59	0.55	441
weighted avg	0.64	0.56	0.57	441

0.1.2 Threshold Adjustment

```
[8]: y_probas = lr_best.predict_proba(X_test)[: , 1]
      precision, recall, thresholds = precision_recall_curve(y_test, y_probas)

      plt.plot(thresholds, precision[:-1], label='Precision')
```

```
plt.plot(thresholds, recall[:-1], label='Recall')
plt.xlabel('Threshold')
plt.ylabel('Score')
plt.title('Precision-Recall vs Threshold')
plt.legend()
plt.show()
```



```
[9]: y_probas = lr_best.predict_proba(X_test)[: , 1] # Probabilities for class 1

# Apply custom threshold
y_probas_custom = (y_probas >= 0.40).astype(int)

print(f'Confusion Matrix: \n{pd.crosstab(y_test, y_probas_custom)}\n')
print(f'Classification Report: \n{classification_report(y_test, \u2192
\u2192y_probas_custom)}\n')
```

```
Confusion Matrix:
col_0      1
Malnourished
0          292
```

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	292
1	0.34	1.00	0.51	149
accuracy			0.34	441
macro avg	0.17	0.50	0.25	441
weighted avg	0.11	0.34	0.17	441

```
/home/codespace/.local/lib/python3.12/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/home/codespace/.local/lib/python3.12/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/home/codespace/.local/lib/python3.12/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

0.1.3 Average precision

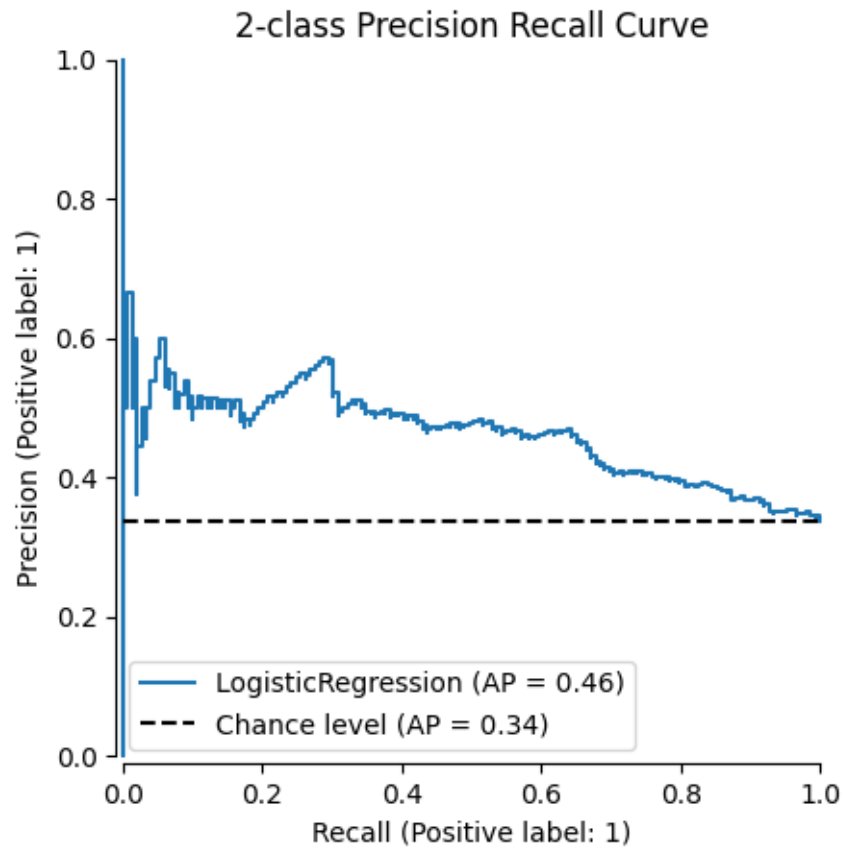
```
[10]: print(f'Average Precision: {average_precision_score(y_test, y_probas)}')
      print(f'Average Precision Adjusted: {average_precision_score(y_test,
      ↪y_probas_custom)}')
```

Average Precision: 0.46229007898515145

Average Precision Adjusted: 0.3378684807256236

```
[11]: display = PrecisionRecallDisplay.from_estimator(
      lr_best,
      X_test,
      y_test,
      name="LogisticRegression",
      plot_chance_level = True,
      despine = True,
      )
      display.ax_.set_title('2-class Precision Recall Curve')
```

```
plt.show()
```



Extract feature coefficient and p-value

```
[12]: import statsmodels.api as sm

# Add intercept to feature set
X_train_with_intercept = sm.add_constant(X_train)

# Fit logistic regression model using statsmodels
logit_model = sm.Logit(y_train, X_train_with_intercept)
result = logit_model.fit()

print(result.summary())
```

Optimization terminated successfully.

Current function value: 0.610003

Iterations 6

Logit Regression Results

=====

Dep. Variable:	Malnurished	No. Observations:	1764
Model:	Logit	Df Residuals:	1727
Method:	MLE	Df Model:	36
Date:	Fri, 11 Apr 2025	Pseudo R-squ.:	0.04629
Time:	14:18:14	Log-Likelihood:	-1076.0
converged:	True	LL-Null:	-1128.3
Covariance Type:	nonrobust	LLR p-value:	1.368e-08

[0.025 0.975]		coef	std err	z	P> z

const		-0.7571	0.391	-1.937	0.053
-1.523	0.009				
Child_age		0.0911	0.055	1.645	0.100
-0.017	0.200				
BMI		-0.0906	0.060	-1.501	0.133
-0.209	0.028				
Children_under5		0.1702	0.085	2.010	0.044
0.004	0.336				
Total_children_ever_born		-0.1533	0.123	-1.242	0.214
-0.395	0.089				
Mother_age_current		0.1033	0.061	1.685	0.092
-0.017	0.223				
Mother_education_1		-0.1013	0.163	-0.621	0.534
-0.421	0.218				
Mother_education_2		-0.1700	0.185	-0.921	0.357
-0.532	0.192				
Mother_education_3		-0.2022	0.368	-0.549	0.583
-0.924	0.519				
Wealth_index_1		-0.1384	0.165	-0.839	0.402
-0.462	0.185				
Wealth_index_2		-0.2578	0.179	-1.439	0.150
-0.609	0.093				
Wealth_index_3		-0.5174	0.203	-2.549	0.011
-0.915	-0.120				
Wealth_index_4		-0.4675	0.252	-1.858	0.063
-0.961	0.026				
Place_residence_rural		0.1872	0.112	1.664	0.096
-0.033	0.408				
Region_madhesh province		0.2761	0.218	1.269	0.204
-0.150	0.702				
Region_bagmati province		0.3521	0.239	1.473	0.141
-0.116	0.821				
Region_gandaki province		0.0037	0.256	0.014	0.989
-0.499	0.506				
Region_lumbini province		0.4246	0.199	2.131	0.033

0.034	0.815				
Region_karnali province		0.5808	0.224	2.588	0.010
0.141	1.021				
Region_sudurpashchim province		0.4493	0.221	2.031	0.042
0.016	0.883				
Religion_buddhist		-0.1108	0.316	-0.351	0.725
-0.729	0.508				
Religion_muslim		0.3018	0.371	0.812	0.417
-0.426	1.030				
Religion_kirat		-0.4792	0.529	-0.905	0.365
-1.517	0.558				
Religion_christian		-0.1316	0.328	-0.402	0.688
-0.774	0.511				
Ethnicity_hill chhetri		-0.2140	0.244	-0.878	0.380
-0.692	0.264				
Ethnicity_terai brahmin/chhetri		0.2350	0.562	0.418	0.676
-0.867	1.337				
Ethnicity_other terai caste		0.5613	0.291	1.927	0.054
-0.010	1.132				
Ethnicity_hill dalit		0.1331	0.269	0.495	0.620
-0.393	0.660				
Ethnicity_terai dalit		0.2447	0.333	0.736	0.462
-0.407	0.897				
Ethnicity_newar		-0.7313	0.538	-1.359	0.174
-1.786	0.323				
Ethnicity_hill janajati		-0.1800	0.270	-0.667	0.505
-0.709	0.349				
Ethnicity_terai janajati		-0.0637	0.289	-0.220	0.826
-0.631	0.503				
Child_sex_female		0.0564	0.105	0.537	0.591
-0.149	0.262				
Father_education_1.0		-0.1059	0.197	-0.539	0.590
-0.491	0.280				
Father_education_2.0		-0.1045	0.213	-0.490	0.624
-0.522	0.313				
Father_education_3.0		-0.1899	0.321	-0.592	0.554
-0.819	0.439				
Health_insurance_yes		-0.0708	0.202	-0.351	0.726
-0.467	0.325				

=====

=====

0.2 Feature selection

0.2.1 Using SelectKBest with f_classif

```
[13]: from sklearn.feature_selection import SelectKBest, f_classif

selector = SelectKBest(f_classif, k=10)
X_new = selector.fit_transform(X_train, y_train)

# Get the names of the selected features
selected_features = X_train.columns[selector.get_support()]

# Print the selected feature names and the shape of the transformed data
print("Selected Features:", selected_features.tolist())
print("Shape of selected features data:", X_new.shape)
```

Selected Features: ['Child_age', 'BMI', 'Children_under5', 'Mother_education_2', 'Wealth_index_3', 'Wealth_index_4', 'Place_residence_rural', 'Region_madhesh province', 'Ethnicity_other terai caste', 'Ethnicity_hill janajati']
Shape of selected features data: (1764, 10)

The SelectKBest method with f_classif scores each feature using the ANOVA F-test and selects the top 18 features based on these scores.

0.3 Optional

0.3.1 ROC curve

```
[14]: y_probas_custom = lr.predict_proba(X_test)[:,:1]
fpr, tpr, _ = metrics.roc_curve(y_test, y_probas_custom)
auc = metrics.roc_auc_score(y_test, y_probas_custom)
plt.plot(fpr,tpr,label="data 1, auc="+str(auc))
plt.legend(loc=4)
plt.show()
```

